

CAPSTONE PROJECT-SMART LIBRARY MANAGER

Abstract:

A Smart Library System is all about making library work easier and smoother. Instead of spending hours writing down details in registers or searching through piles of records, everything is stored digitally. With this system, librarians can quickly add new books, update details, or check if a book is available. Students or members can easily borrow and return books without confusion, while the system keeps track of everything in the background. This not only saves time but also avoids mistakes and keeps the library well-organized. In short, it's a simple and smart way to run a library in today's digital world.

The requirements of our project are:

- Classes
- Libraries
- Methods
- Modules
- Design Patterns
- Polymorphism
- Logs
- Delimiter
- Text files

Classes which we have used in our project are:

a) **Book class**-this represents a single library book.

Attributes used are:

- `book_id`: A special number assigned to every book.
- `title`: The book's title.
- `author`: The book's author.
- `ISBN`: International Standard Book Number.
- `Boolean`: available (False = issued, True = available).

CAPSTONE PROJECT-SMART LIBRARY MANAGER

Techniques which we have implemented is->

Book data is converted into a line for file storage using the `to_line()` function.

We have used ->

`from_line(line)` which returns a Book object from a stored line.

`__str__()` which is used to provide book details that are readable by humans.

Class Book- Code

class Book:

```
def __init__(self, book_id: str, title: str, author: str, isbn: str, available: bool = True):
```

```
    self.book_id = book_id
```

```
    self.title = title
```

```
    self.author = author
```

```
    self.isbn = isbn
```

```
    self.available = available
```

```
def to_line(self) -> str:
```

```
    return "|".join([self.book_id, self.title, self.author, self.isbn, "1" if self.available else "0"]) +  
    "\n"
```

```
@staticmethod
```

```
def from_line(line: str) -> "Book":
```

```
    parts = line.strip().split("|")
```

```
    return Book(parts[0], parts[1], parts[2], parts[3], parts[4] == "1")
```

```
def __str__(self) -> str:
```

```
    status = "Available" if self.available else "Borrowed"
```

```
    return f"[{self.book_id}] {self.title} by {self.author} (ISBN: {self.isbn}) — {status}"
```

Logic used in the code:

The Book class is designed to handle all details about a book.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

- When we create a book, it stores its ID, title, author, ISBN, and whether it's available or borrowed.
- The `to_line` method turns the book's details into a single line of text so it can be saved in a file.
- The `from_line` method does the opposite — it reads a line from the file and recreates the book object.
- The `__str__` is a dunder method showing details like title, author, ISBN, and status.
- We have used `return` because the function gives back the formatted book data string to the part of the program that needs it (like when writing to a file) and that string can be reused
- We used the delimiter "|" to separate different pieces of book information clearly inside a single line when saving to a file.

b) **Class of Members**-represents a patron of the library.

Attributes used are:

- `member_id`: Each member's unique identifier.
- `name`: The member's name.
- `borrowed`: A list of book IDs that have been borrowed.

Techniques which we have implemented is->

- member data we have added it into a line for file storage using the `to_line()` function.
- `from_line(line)` is used which returns a Member object from a stored line.
- We have used `__str__()` which provides member details that are readable by humans.

Class Member-Code

class Member:

```
def __init__(self, member_id: str, name: str, borrowed: Optional[List[str]] = None):  
    self.member_id = member_id  
    self.name = name  
    self.borrowed = borrowed or []
```

```
def to_line(self) -> str:  
    borrowed_str = ",".join(self.borrowed)  
    return "|".join([self.member_id, self.name, borrowed_str]) + "\n"
```

CAPSTONE PROJECT-SMART LIBRARY MANAGER

```
@staticmethod
```

```
def from_line(line: str) -> "Member":
```

```
    parts = line.strip().split(" | ")
```

```
    borrowed = parts[2].split(",") if parts[2] else []
```

```
    return Member(parts[0], parts[1], borrowed)
```

```
def __str__(self) -> str:
```

```
    return f"[{self.member_id}] {self.name} - Borrowed: {' | '.join(self.borrowed) if self.borrowed else 'None'}"
```

Logic used in the code:

- When a new library member is added, they're given a membership ID, name, and a list of borrowed books (which starts empty if they haven't taken any).
- Whenever the system needs to save member details into a file, it turns their information into a single line of text – almost like writing their record on a library card.
- Later, if the program is restarted, the system can read that line back from the file and rebuild the member's profile exactly as it was before.
- When we want to see a member's details, the system shows their ID, name, and the books they currently have borrowed. If they haven't borrowed anything, it simply shows "None."

c)**Library Class**-Acts as the controller and manager of the system.

Attributes we have used are:

- books: A dictionary that stores all Book objects.
- members: A dictionary that stores all Member objects.

Methods we have used are:

load(): Reads data from files like books.txt and members.txt.

save(): Saves the current data back to files.

add_book(): Adds a new book to the system.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

remove_book(): Removes a book if it is available.

register_member(): Adds a new member.

borrow_book(): Allows a member to borrow a book if it is available.

return_book(): Handles the return of a borrowed book.

list_books(): Displays all books.

list_members(): Displays all members.

Class Library-Code

```
class Library:
    def __init__(self):
        self.books: Dict[str, Book] = {}
        self.members: Dict[str, Member] = {}

    def load(self):
        if os.path.exists("books.txt"):
            with open("books.txt", "r") as f:
                for line in f:
                    book = Book.from_line(line)
                    self.books[book.book_id] = book
        if os.path.exists("members.txt"):
            with open("members.txt", "r") as f:
                for line in f:
                    member = Member.from_line(line)
                    self.members[member.member_id] = member

    def save(self):
        with open("books.txt", "w") as f:
            for book in self.books.values():
                f.write(book.to_line())

        with open("members.txt", "w") as f:
            for member in self.members.values():
                f.write(member.to_line())
```

CAPSTONE PROJECT-SMART LIBRARY MANAGER

Logic used in the code:

- When the program starts, it creates empty storage for books and members.
- The load method checks if books.txt and members.txt already exist.
- If the files exist, the system reads them and restores all books and members into memory.
- This makes sure the library remembers everything from the previous session.
- When changes are made (like adding or borrowing), the save method updates the files with the new details.
- This ensures that all data is kept safe and nothing is lost, even after the program is closed.

Libraries Used in the Code:

1. os

- Helps the program check if files already exist before trying to load them.
- This makes sure that data (like books and members) is not lost and can be reloaded when the program starts again.

2. typing

- Used to add type hints (like Dict, List, Optional) in the code.
- This doesn't change how the program runs but makes the code easier to read, understand, and maintain

Methods Used in the Code:

1. File Handling

load(), save(), to_line(), from_line() : These methods take care of reading data from files and saving it back, so book and member information is always up to date.

2. Admin Operations

add_book(), remove_book(), register_member() : These methods are for the librarian/admin to manage the library by adding or removing books and registering new members.

3. User Operations

CAPSTONE PROJECT-SMART LIBRARY MANAGER

borrow_book(), return_book() :These methods let library members borrow books (if available) and return them when done.

4. Utility Methods

list_books(), list_members(), __str__() :These help in showing the list of all books or members in the library and provide a clean way to display information.

Code - Admin Methods

```
def add_book(self, book_id: str, title: str, author: str, isbn: str):  
  
    self.books[book_id] = Book(book_id, title, author, isbn)  
  
def remove_book(self, book_id: str):  
  
    if book_id in self.books:  
  
        del self.books[book_id]  
  
def register_member(self, member_id: str, name: str):  
  
    self.members[member_id] = Member(member_id, name)
```

Logic of the Code-

- add_book() → Inserts a new book.
- remove_book() → Delete a book if it exists.
- register_member() → Add new member to system.

Code - User Methods

```
def borrow_book(self, member_id: str, book_id: str):  
  
    if member_id in self.members and book_id in self.books:  
  
        member = self.members[member_id]  
  
        book = self.books[book_id]
```

CAPSTONE PROJECT-SMART LIBRARY MANAGER

```
if book.available:

    book.available = False

    member.borrowed.append(book_id)

def return_book(self, member_id: str, book_id: str):

    if member_id in self.members and book_id in self.books:

        member = self.members[member_id]

        book = self.books[book_id]

        if book_id in member.borrowed:

            book.available = True

            member.borrowed.remove(book_id)
```

Logic of the code

borrow_book() →

- Checks if the book exists and is available.
- Change book status → available = False.
- Add book ID to member's borrowed list.

return_book() →

- Check if a member actually borrowed the book.
- Change book status → available = True.
- Remove book ID from member's borrowed list.

Code - Utility Methods

```
def list_books(self):

    for book in self.books.values():

        print(book)

def list_members(self):
```

CAPSTONE PROJECT-SMART LIBRARY MANAGER

```
for member in self.members.values():
```

```
    print(member)
```

Logic of the code

list_books() → Show all books with availability.

list_members() → Show all members and what they borrowed.

Approach of our Project

The main goal of using this approach is to digitalize library management using Python and file storage. Instead of manually recording books and members, the system automates these tasks through classes and text files.

We have used:

- **Object-Oriented Design**
 - Book class: Represents book details.
 - Member class: Represents library members.
 - Library class: Manages books, members, and transactions.
- **File-Based Persistence**
 - Uses books.txt and members.txt to store and retrieve data.
 - Ensures data is saved between program runs.
- **Admin and User Operations**
 - Admin: Add or remove books, register members.
 - User: Borrow or return books.
- **CLI Menu-Driven Program**
 - Text-based menu allows users to interact with the system.

Design of the Project

CAPSTONE PROJECT-SMART LIBRARY MANAGER

The smart library system is designed to be simple and easy to use. It has a few main parts that work together to keep the library running smoothly.

- **Books** are stored with details like title, author, and whether they are available or already borrowed.
- **Members** have their own records showing who they are and which books they've borrowed.
- The **Library** brings everything together. It manages books, members, and all the actions like borrowing, returning, and saving data.

All this information is saved in two text files—one for books and one for members—so that even if the program is closed, nothing is lost.

The system is used through a **simple menu** that appears on the screen. The user just picks what they want to do, like adding a book, borrowing one, or checking the list of members. The system then takes care of the rest.

How the flow of the code works : Step by Step

- **When the program starts**
 - It first loads the data from two files: one for books and one for members.
 - This way, whatever was saved earlier comes back into the system, so nothing is lost.
- **While using the program**
 - A menu shows up, and the user simply picks what they want to do. For example:
 - Add a new book → the book is created and saved.
 - Remove a book → the system checks and deletes it if possible.
 - Register a member → a new member is added and saved.
 - Borrow a book → the system marks it as borrowed and links it to the member.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

- Return a book : the book becomes available again and is removed from the member's borrowed list.
- List books : shows all books and whether they're available or not.
- List members : shows all members and the books they've borrowed.

From the above list the user opts one of their choice of interest and performs the necessary operation.

- **When the program closes**
 - All the changes made are saved back into the files.
 - This means the library is always up to date the next time you open the program.

1.File Handling

- The system uses two files — **books.txt and members.txt** — to store information about the books added,removed,updated and borrowed.
- These files act like the library's notebooks that keep records safe.
- When the program starts, it reads the files and loads all the saved books and members back into memory.
- Any changes (like adding/removing books or borrowing/returning) are first updated in memory.
- When saving, the updated details are written back into the files.
- This makes sure data is not lost, even after the program is closed or restarted.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

example in our code :

```
for line in f:
    if line.strip():
        b = Book.from_line(line)
        ➤ Read each line (like one book/member record).
        ➤ strip() cleans extra spaces/newlines.
        ➤ Converts that line into a Book or Member object.
```

2.Usage of UTF-8?

UTF-8 is a method that allows computers to agree on how to store and read letters, numbers, symbols, and even emojis in a consistent format. If we don't specify UTF-8, different systems may save the same text in various ways.

IN our code we have used->`open("file.txt", "r", encoding="utf-8")`

It means that we can read and write this file using the universal language for text so everyone sees the same thing.

3. Use of `__init__` constructor method

`__init__` is a setup whenever we are creating something new

Here in ->`b = Book("B1", "Python", "AuthorX", "12345")`

`__init__` is what automatically fills in all the details of that Book, like writing down its ID, title, and more. In the Library class, it also ensures there are empty shelves (dictionaries) ready for books and members. Additionally, it loads existing data right away.

4.Why have we used `@staticmethod`?

A static method is simply a regular function we include in a class for better organization. In our project , `from_line()` doesn't need to modify any existing object. It just takes some text and creates a new Book or Member. Instead of writing a separate function, we place it inside the class with `@staticmethod`,it allows us to call `Book.from_line(line)` directly, without having to create a Book first.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

5.What's the strip() method, and why did we use it?

```
parts = line.strip().split(DELIM)
```

and:

```
if line.strip():
```

strip() cleans up strings by removing extra spaces or hidden characters, like newlines, from the start or end. When we read lines from a file, they almost always have a \n at the end.

By using strip(), we ensure we're working with clean, trimmed text. This stops annoying bugs, like empty lines being treated as real data or IDs having random spaces in them.

6. Why did we use the .txt file in our project?

books.txt keeps details of books and members.txt keeps details of members . Every time you add, borrow, or return, the program updates the notebook so data stays safe. It's a simple way to give the program a memory without using a database.

7.Usage of cli and how it works.

The CLI (Command Line Interface) is like a menu on the blackboard where you type what you want the library system to do.Instead of clicking buttons , we can simply type numbers between 1-6 to perform the preferred action.It makes the program easy to use without a fancy GUI.

8. Exception Handling

- raise ValueError("Corrupt book line: " + line) ->If the line doesn't contain exactly 5 pieces, the program immediately stops this operation and throws an error saying the line is corrupt. This is exception handling — it ensures you don't create a Book object with incomplete or messed-up data.
- raise ValueError("Corrupt member line: " + line) -> raise ValueError(...) If the line doesn't have exactly 3 parts, we raise an exception instead of silently continuing with bad data. ValueError is a built-in Python exception used when data isn't in the expected format.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

9. Purpose of using ->

- The arrow -> just means “this function will return this type of thing”.
- It makes the code easier to read and understand.
- In our code, example:

```
def from_line(line: str) -> "Book":
```

- Input: line is a string (str).
- Output: the function gives back a Book object (Book).

10. Can we save the main function into another file. Will that work?

No, it won't directly work. Because those functions depend on other parts of the project like `library.py` and [main.py](#) where,

- `library.py` will have all the classes and logic (Book, Member, Library).
- In `main.py`, you must import those classes/modules to use them.

Without importing, Python won't know where Library (or Book/Member) is defined. We need to import the classes/modules to make both files work together.

11. Why delimiter is used?

Delimiter is a special character (like | or ,) it is used to separate fields in a text file.

Example in `books.txt`:

```
B101|Python Basics|Abhi|9781234567890|1
```

In our project we have used | as delimiter, so the program can easily split data into parts.

CAPSTONE PROJECT-SMART LIBRARY MANAGER

Conclusion

The Smart Library Manager is a simple project that shows how Python can be used to make library work easier. It keeps track of books and members in an organized way, saves the data so nothing is lost, and gives users a clear menu to do their tasks without confusion.

In the future, this system can grow bigger and smarter. We could use a database instead of text files, add a proper app-like interface, and include extra features like book search, due dates, and fines—making it feel just like a real library system.